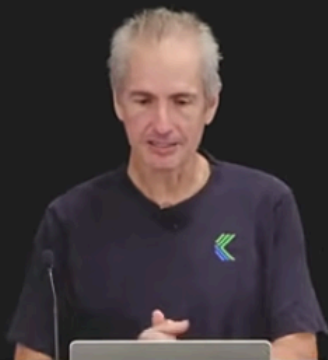


Building C++20 Modules

The Rest of the Story

VITO GAMBERINI



CMake import std

```
[1/14] "CLANG_ROOT/bin/clang-scan-deps" -format=p1689 -- CLANG_ROOT/bin/clang++ -stdlib=libc++ -std=c++11
[2/14] "CLANG_ROOT/bin/clang-scan-deps" -format=p1689 -- CLANG_ROOT/bin/clang++ -ICLANG_ROOT/bin/./lib/x86_64-unknown-linux-gnu/./../share/libc++/v1
[3/14] "CLANG_ROOT/bin/clang-scan-deps" -format=p1689 -- CLANG_ROOT/bin/clang++ -stdlib=libc++ -std=c++11
[4/14] CMAKE_ROOT/bin/cmake -E cmake_ninja_dyndep --tdi=CMakeFiles/main.dir/CXXDependInfo.json --lang=C
[5/14] "CLANG_ROOT/bin/clang-scan-deps" -format=p1689 -- CLANG_ROOT/bin/clang++ -ICLANG_ROOT/bin/./lib/x86_64-unknown-linux-gnu/./../share/libc++/v1
[6/14] CMAKE_ROOT/bin/cmake -E cmake_ninja_dyndep --tdi=CMakeFiles/__cmake_cxx23.dir/CXXDependInfo.json --lang=C
[7/14] CMAKE_ROOT/bin/cmake -E cmake_ninja_dyndep --tdi=CMakeFiles/uses_std.dir/CXXDependInfo.json --lang=C
[8/14] CLANG_ROOT/bin/clang++ -stdlib=libc++ -std=gnu++20 -MD -MT CMakeFiles/main.dir/main.cxx.o -MF CMakeFiles/main.dir/main.cxx.o.d -c CMakeFiles/main.dir/main.cxx
[9/14] CLANG_ROOT/bin/clang++ -ICLANG_ROOT/bin/./lib/x86_64-unknown-linux-gnu/./../share/libc++/v1 -std=libc++ -std=gnu++20 -MD -MT CMakeFiles/main.dir/main.cxx.o -MF CMakeFiles/main.dir/main.cxx.o.d -c CMakeFiles/main.dir/main.cxx
[10/14] CLANG_ROOT/bin/clang++ -ICLANG_ROOT/bin/./lib/x86_64-unknown-linux-gnu/./../share/libc++/v1 -std=libc++ -std=gnu++20 -MD -MT CMakeFiles/main.dir/main.cxx.o -MF CMakeFiles/main.dir/main.cxx.o.d -c CMakeFiles/main.dir/main.cxx
[11/14] : && CMAKE_ROOT/bin/cmake -E rm -f lib__cmake_cxx23.a && CLANG_ROOT/bin/llvm-ar qc lib__cmake_cxx23.a && CMAKE_ROOT/bin/cmake -E mv lib__cmake_cxx23.a lib__cmake_cxx23.a
[12/14] CLANG_ROOT/bin/clang++ -stdlib=libc++ -std=gnu++20 -MD -MT CMakeFiles/uses_std.dir/uses_std.cxx.o -MF CMakeFiles/uses_std.dir/uses_std.cxx.o.d -c CMakeFiles/uses_std.dir/uses_std.cxx
[13/14] : && CMAKE_ROOT/bin/cmake -E rm -f libuses_std.a && CLANG_ROOT/bin/llvm-ar qc libuses_std.a && CMAKE_ROOT/bin/cmake -E mv libuses_std.a libuses_std.a
[14/14] : && CLANG_ROOT/bin/clang++ -stdlib=libc++ -Wl,-rpath,CLANG_ROOT/lib/x86_64-unknown-linux-gnu -std=libc++ -std=gnu++20 -MD -MT CMakeFiles/uses_std.dir/uses_std.cxx.o -MF CMakeFiles/uses_std.dir/uses_std.cxx.o.d -c CMakeFiles/uses_std.dir/uses_std.cxx
```

Question

main.cpp	-DDEBUG	-Iinclude
render.cpp	-DENABLE_VULKAN	-Irender
math.cpp	-DUSE_SIMD	-Imath
util.cpp	-DUSE_STD_FORMAT	-Iutil

```
c++ -c main.cpp -o main.o -DDEBUG -Iinclude
c++ -c render.cpp -o render.o -DENABLE_VULKAN -Irender
c++ -c math.cpp -o math.o -DUSE_SIMD -Imath
c++ -c util.cpp -o util.o -DUSE_STD_FORMAT -Iutil
```

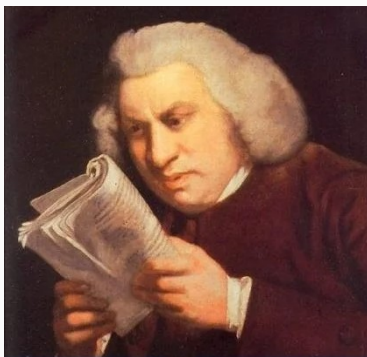
```
c++ main.o render.o math.o util.o -o prog
```

Question

module interface units	main.cpp	-DDEBUG	-Iinclude
	render.cppm	-DENABLE_VULKAN	-Irender
	math.cppm	-DUSE_SIMD	-Imath
	util.cppm	-DUSE_STD_FORMAT	-Iutil

Question

main.cpp		-DDEBUG	-std=c++20
render.cppm		-DENABLE_VULKAN	-std=c++20
math.cppm		-DUSE_SIMD	-std=c++23
util.cppm		-DUSE_STD_FORMAT	-std=c++26



Topics

- How to build C++20 modules
- Specific command examples and their purpose
- The general consensus method
- Unsolved problems
- Misconceptions and Pitfalls

Before Modules

Translation Unit

```
#include <vector>
#include <string>
#include <iostream>

int add(int a, int b) {
    return a + b;
}

double mul(double a, double b) {
    return a * b;
}

int clamp(int v, int lo, int hi) {
    return v < lo ? lo : v;
}
```

After Modules

Translation
Unit which
is not a
Module Unit

Header
Unit

Named Module Units

```
#include "math.h"
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v = range(5);
    int total = sum(v);
    std::cout << total << "\n";
    std::cout << greet() << "\n";
    for (int x : v) {
        std::cout << x << "\n";
    }
    return 0;
}
```

```
#ifndef MATH_H
#define MATH_H

#include <string>
#include <vector>

int square(int x);
int cube(int x);
int min(int a, int b);
int max(int a, int b);
std::string greet();
std::vector<int> range(int n);
int sum(const std::vector<int> v);

#endif
```

```
export module math;

export int add(int a, int b) {
    return a + b;
}

export int sub(int a, int b) {
    return a - b;
}

export int mul(int a, int b) {
    return a * b;
}

export int div(int a, int b);
```

```
module math;

int add(int a, int b) {
    return a + b;
}

int sub(int a, int b) {
    return a - b;
}

int mul(int a, int b) {
    return a * b;
}

int helper(int x);
```

```
export module math:core;

export int square(int x);
export int cube(int x);
export int min(int a, int b);
export int max(int a, int b);
export int abs(int x);
export int gcd(int a, int b);
export int lcm(int a, int b);
export int pow(int b, int e);
export int clamp(int v, int lo, int hi);
export int sign(int x);
export int negate(int x);
export int inc(int x);
export int dec(int x);
```

```
module math:core;

int square(int x) {
    return x * x;
}

int cube(int x) {
    return x * x * x;
}

int min(int a, int b) {
    return a < b ? a : b;
}

int max(int a, int b);
```

P1838: Modules User-Facing Lexicon and File Extensions

Module Declarations

10.1 Module units and purviews

[module.unit]

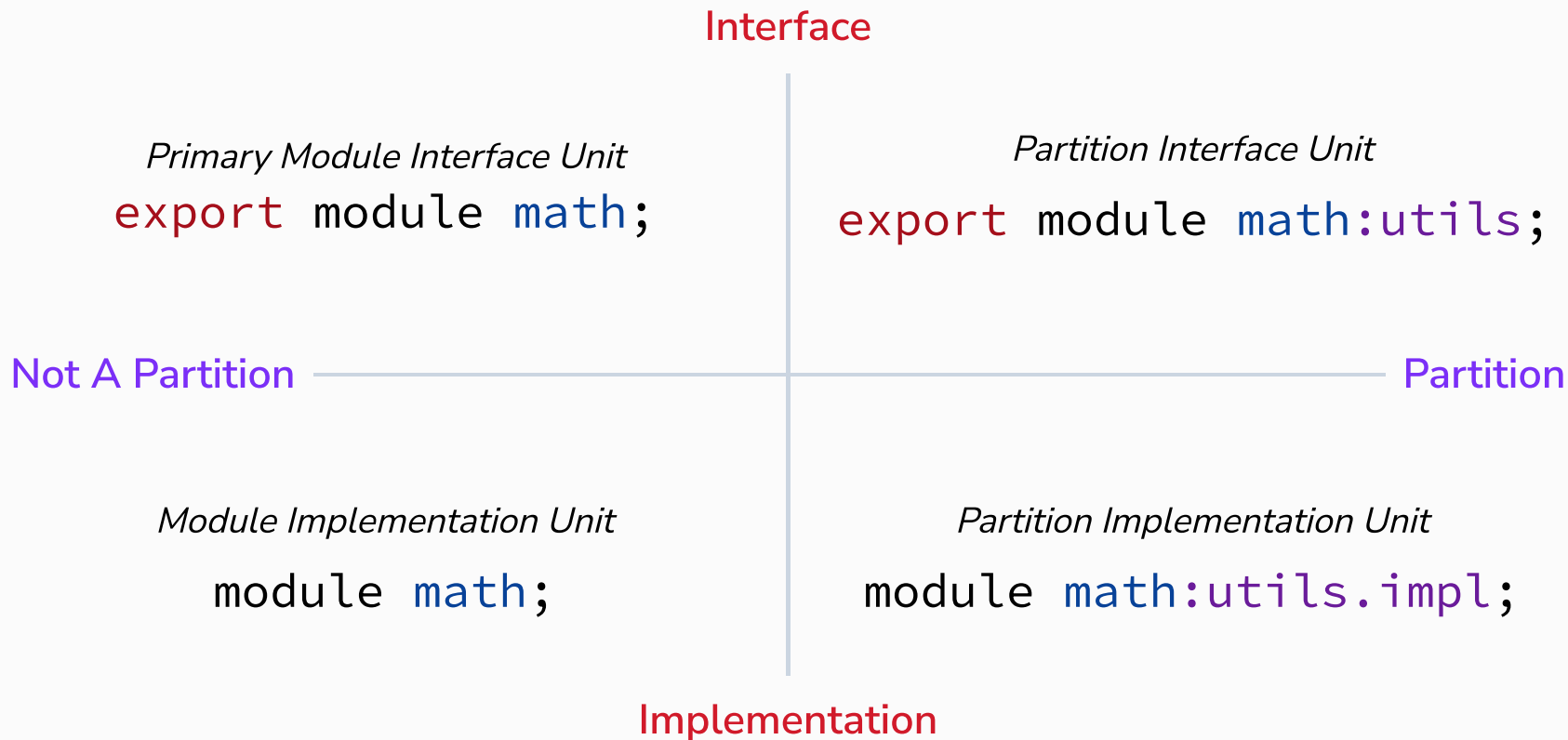
module-declaration:

```
export-keywordopt module-keyword module-name  
module-partitionopt ;
```

```
export module math:utils;
```

A *module partition* is a module unit whose *module-declaration* contains a module-partition.

Module Units



import

Module Units

Interface

`export module math;`

- Importable
- Necessarily reachable

`export module math:utils;`

- Importable within module
- Necessarily reachable

Not A Partition

Partition

`module math;`

- Not importable
- Not reachable

`module math:utils.impl;`

- Importable within module
- Implementation-defined reachability

Implementation

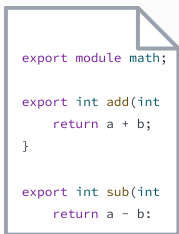
Let's Build a PMIU

```
export module math;
```

```
export auto add(auto a, auto b) {  
    return a + b;  
}
```

```
import math; ? ? ?
```

```
int main() {  
    return add(1, 2);  
}
```



math.cppm



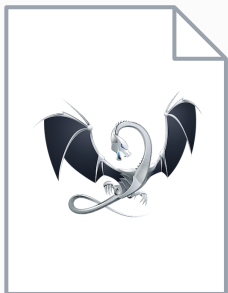
math.o



math.bmi

Built Module Interface

Clang



PCM

LLVM Bitcode

`clang::ASTWriter`



`llvm::BitstreamWriter`

GCC



GCM

ELF / ELRoND

`trees_out`



`elf_out`

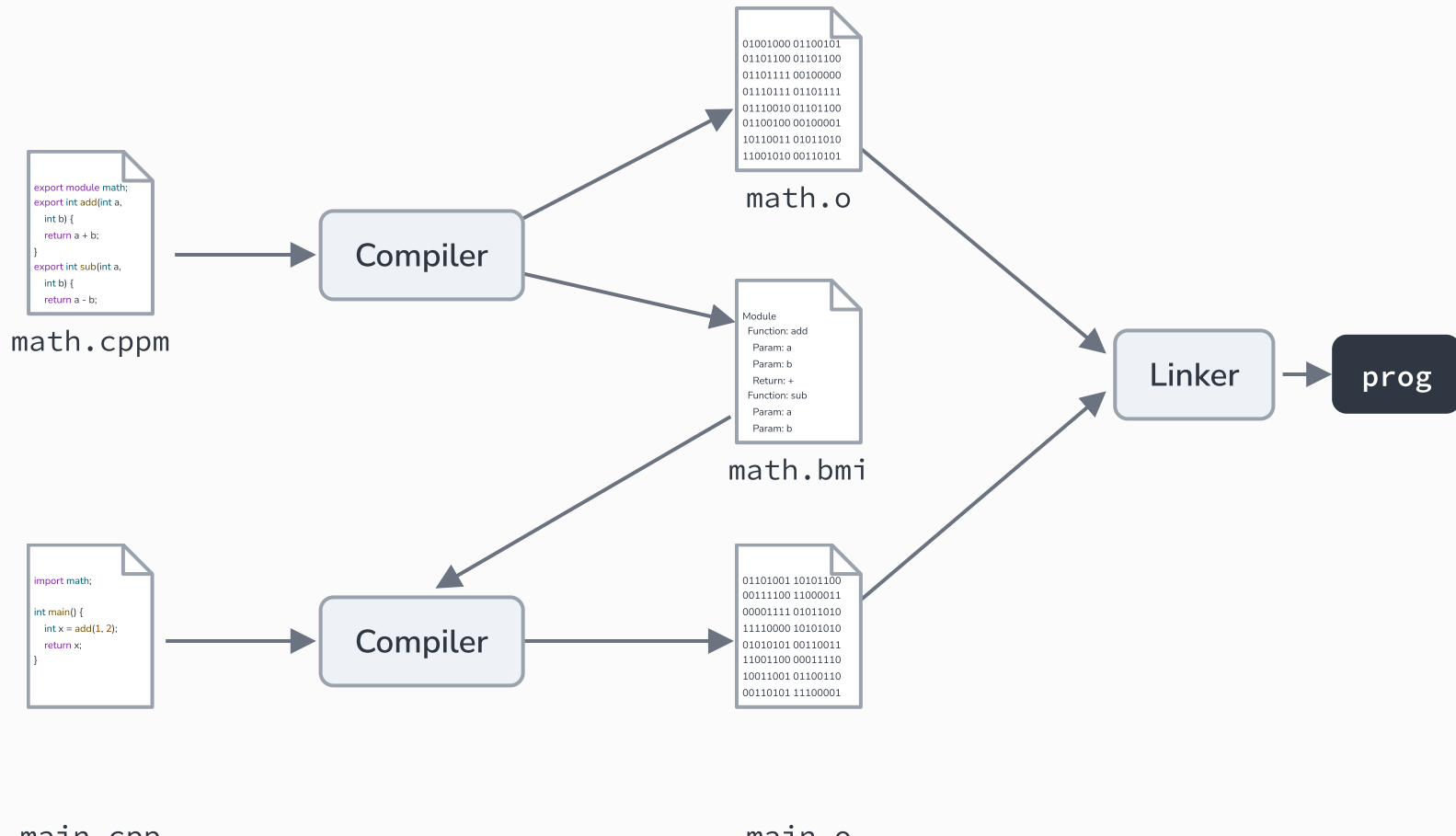
MSVC



IFC

IFC (file format)

Build & Consume a PMIU



Let's Build a PMIU

```
clang++ -std=c++20 -c math.cppm -o math.o -fmodule-output=math.bmi
```

```
clang++ -std=c++20 -fmodule-file=math=math.bmi -c main.cpp -o main.o
```

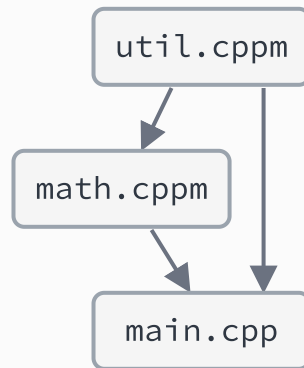
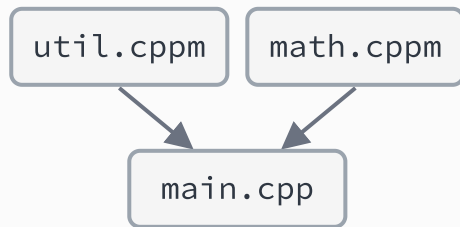
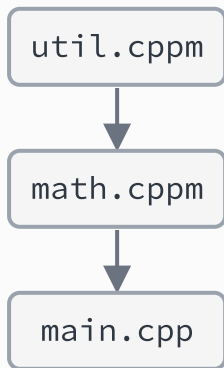
```
clang++ math.o main.o -o prog
```

Let's Build Two PMIUs

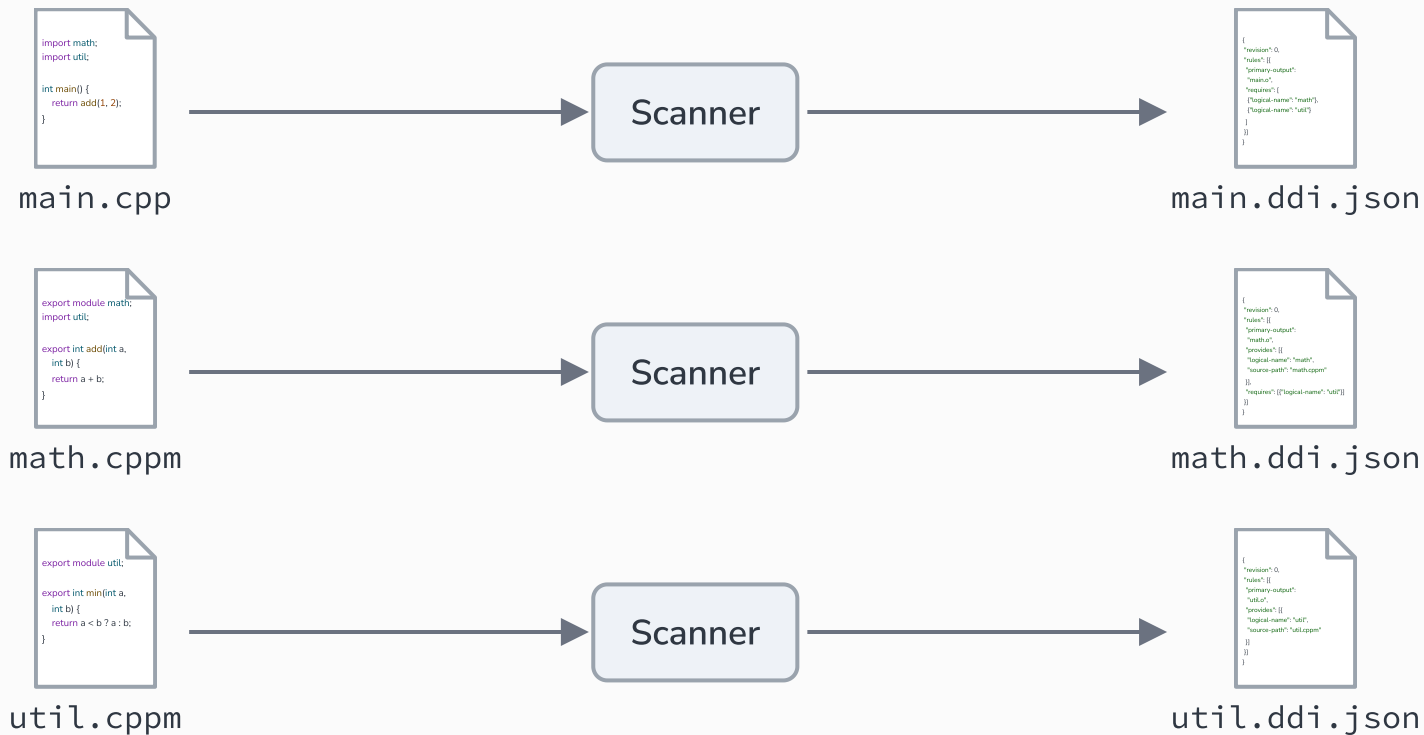
main.cpp		-std=c++20
----------	--	------------

math.cppm		-std=c++20
-----------	--	------------

util.cppm		-std=c++20
-----------	--	------------

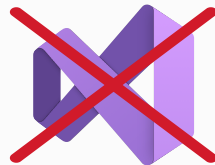


The Scanner



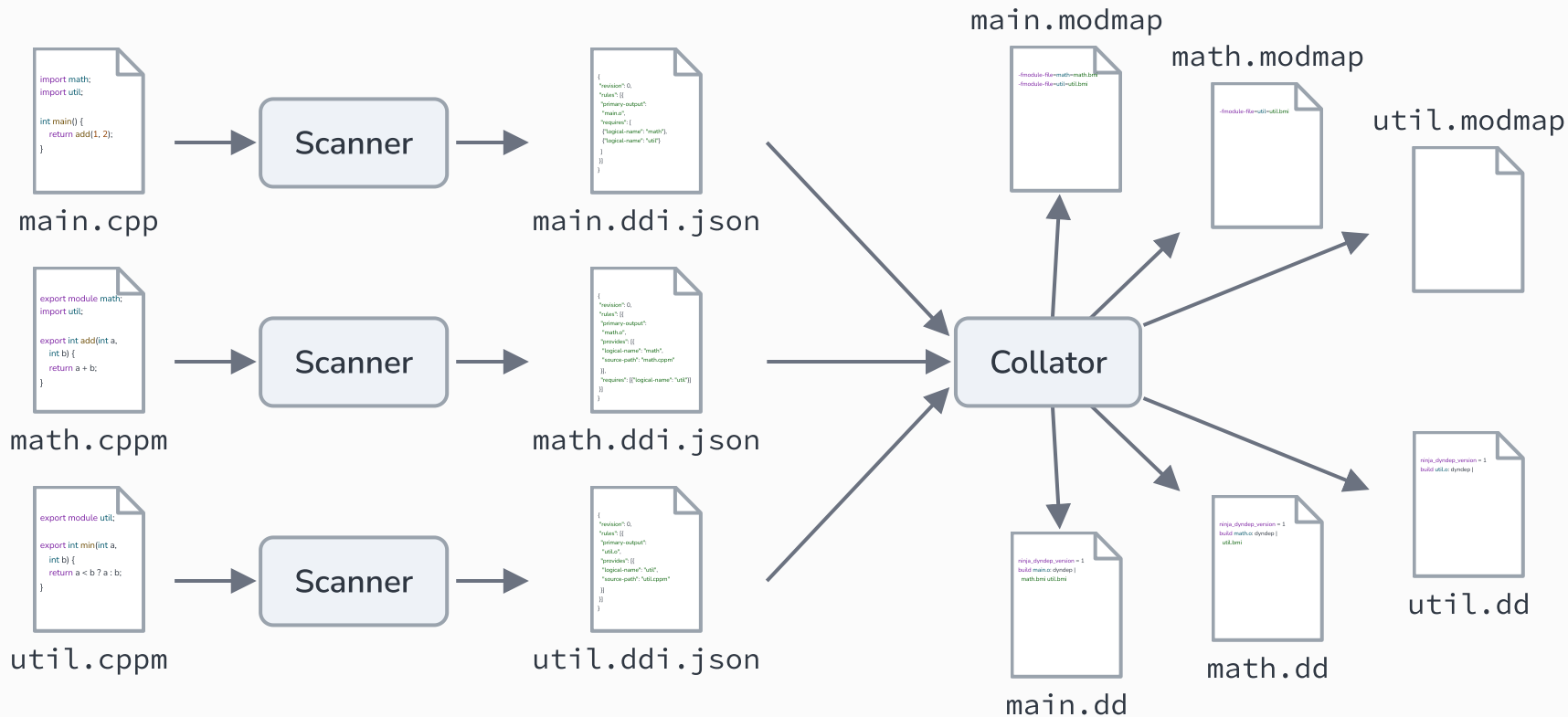
The Scanner Format

```
{  
  "revision": 0,  
  "rules": [{  
    "primary-output": "math.o",  
    "provides": [{  
      "is-interface": true,  
      "logical-name": "math",  
      "source-path": "math.cppm"  
    }],  
    "requires": [{"logical-name": "util"}]  
  }],  
  "version": 1  
}
```



P1689: Format for describing dependencies of source files

The Collator



Module Maps

```
-fmodule-file=math=math.bmi  
-fmodule-file=util=util.bmi
```

main.modmap

```
-fmodule-file=util=util.bmi
```

math.modmap

util.modmap

```
clang++ -std=c++20 @main.modmap -c main.cpp -o main.o
```

```
clang++ -std=c++20 -fmodule-file=math=math.bmi -fmodule-file=util=util.bmi -c main.cpp -o main.o
```

P1184: A Module Mapper

DynDeps

```
ninja_dyndep_version = 1  
build main.o: dyndep |  
    math.bmi util.bmi
```

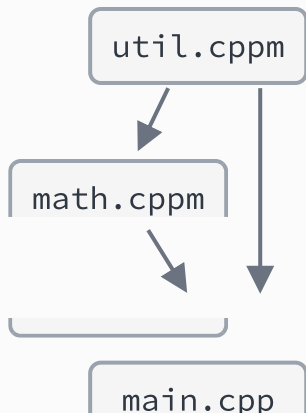
main.dd

```
ninja_dyndep_version = 1  
build math.o: dyndep |  
    util.bmi
```

math.dd

```
ninja_dyndep_version = 1  
build util.o: dyndep |
```

util.dd



Dear Programmer,

Please build the modules in this order:

1. util.cppm
2. math.cppm

3. main.cpp

Sincerely,

A Quick Note About Scanners

```
clang-scan-deps -format=p1689
```

```
g++ -fdeps-format=p1689r5
```

```
cl.exe -scanDependencies
```

```
#ifdef __clang__  
import clang-specific-module;  
#elif __GNUC__ > 15  
import modern-gcc-module;  
#else  
import fallback-module;  
#endif
```

Let's Build Two PMIUs

```
1 clang-scan-deps -format=p1689 -std=c++20 -c main.cpp -o main.ddi.json
```

```
1 clang-scan-deps -format=p1689 -std=c++20 -c math.cppm -o math.ddi.json
```

```
1 clang-scan-deps -format=p1689 -std=c++20 -c util.cppm -o util.ddi.json
```

```
2 collate main.ddi.json math.ddi.json util.ddi.json
```

```
4 clang++ -std=c++20 @math.modmap -c math.cppm -o math.o -fmodule-output=math.bmi
```

```
3 clang++ -std=c++20 @util.modmap -c util.cppm -o util.o -fmodule-output=util.bmi
```

```
5 clang++ -std=c++20 @main.modmap -c main.cpp -o main.o
```

```
6 clang++ math.o util.o main.o -o prog
```

Let's Build Two PMIUs, The Remix

main.cpp		-std=c++20
math.cppm		-std=c++23
util.cppm		-std=c++20

On Built Module Interface Compatibility

or, Testing a bunch of build systems
which claim to support modules

<https://blog.vito.nyc/posts/bmi-compatibility/>

The Problem

```
-std=c++23
```

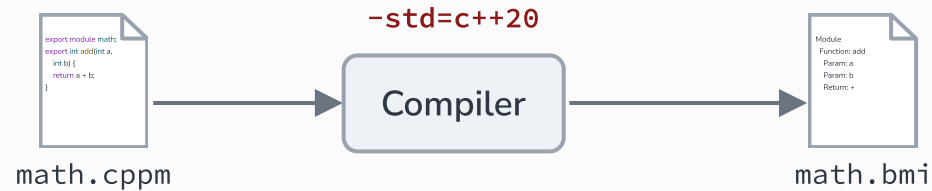
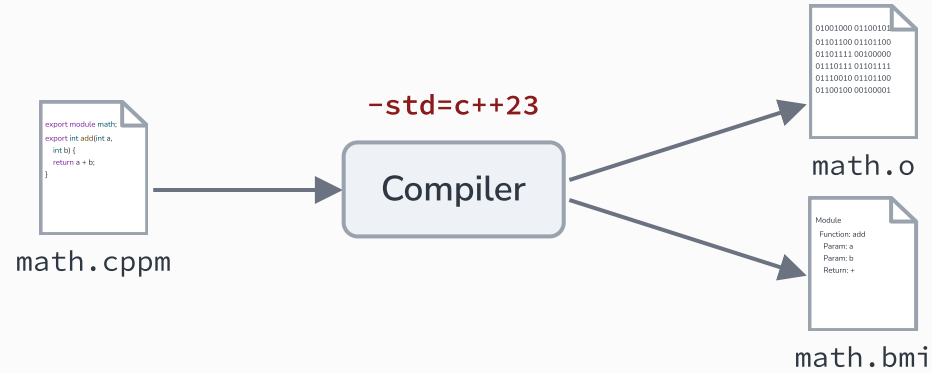
```
#include <print>  
#include "math.hpp"
```

```
int main() {  
    int result = add(1, 2);
```

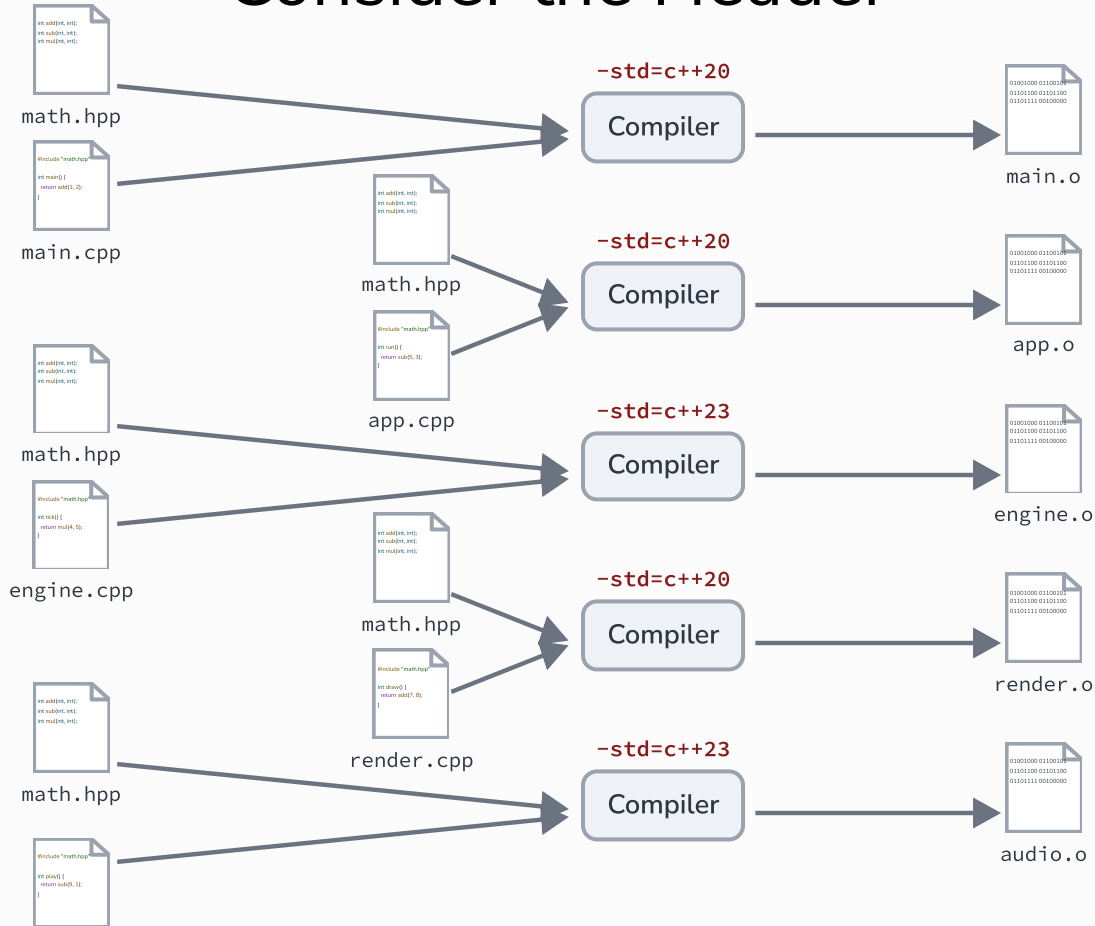
```
-std=c++20
```

```
    std::println("result = {}", result);  
    return 0;  
}
```

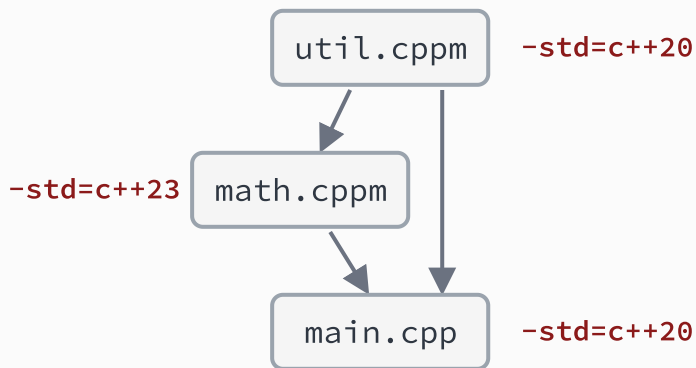
BMI Only



Consider the Header



A Better Collator



Dear Programmer,

Please consider the following dependency information:

- `util.cppm` has no dependencies
- `math.cppm` depends on `util`
- `main.cpp` depends on `math` and `util`

Sincerely,

Let's Build Two PMIUs, The Remix

```
1 clang-scan-deps -format=p1689 -std=c++20 -c main.cpp -o main.ddi.json
1 clang-scan-deps -format=p1689 -std=c++23 -c math.cppm -o math.ddi.json
1 clang-scan-deps -format=p1689 -std=c++20 -c util.cppm -o util.ddi.json
2 collate main.ddi.json math.ddi.json util.ddi.json
3 clang++ -std=c++23 --precompile @util23.modmap util.cppm -o util23.bmi
4 clang++ -std=c++20 --precompile @math20.modmap math.cppm -o math20.bmi
3 clang++ -std=c++20 @util20.modmap -c util.cppm -o util.o -fmodule-output=util20.bmi
4 clang++ -std=c++23 @math23.modmap -c math.cppm -o math.o -fmodule-output=math23.bmi
5 clang++ -std=c++20 @main20.modmap -c main.cpp -o main.o
6 clang++ math.o util.o main.o -o prog
```

Unsolved Problem in Computer Science #1

```
clang-scan-deps -format=p1689 -std=c++20 -c util.cppm -o util.ddi.json
```

```
clang++ -std=c++23 --precompile @util23.modmap util.cppm -o util23.bmi
```

```
clang++ -std=c++20 @util20.modmap -c util.cppm -o util.o -fmodule-output=util20.bmi
```

```
#if __cplusplus >= 202302L
import std;
#endif
```

Unsolved Problem in Computer Science #2

-O2	-fno-rtti	-pthread
-fno-exceptions	-fomit-frame-pointer	-fms-extensions
-g	-funsigned-char	-fsanitize=address
-fchar8_t	-pipe	-fdelayed-template-parsing
-fPIC	-fno-math-errno	-fstack-protector-strong
-fshort-enums	-march=native	-fvisibility=hidden
-gdwarf-5	-ffast-math	-fno-threadsafe-statics

Let's Build Two PMIUs, The Finale

main.cpp

| -std=c++20

math.cppm

| -std=c++23 -DUSE_SIMD

util.cppm

| -std=c++20 -DUSE_STD_FORMAT

Let's Build Two PMIUs, The Finale

```
1 clang-scan-deps -format=p1689 -std=c++20 -c main.cpp -o main.ddi.json
1 clang-scan-deps -format=p1689 -std=c++23 -DUSE_SIMD -c math.cppm -o math.ddi.json
1 clang-scan-deps -format=p1689 -std=c++20 -DUSE_STD_FORMAT -c util.cppm -o util.ddi.json
2 collate main.ddi.json math.ddi.json util.ddi.json
3 clang++ -std=c++23 -DUSE_STD_FORMAT --precompile @util23.modmap util.cppm -o util23.bmi
4 clang++ -std=c++20 -DUSE_SIMD --precompile @math20.modmap math.cppm -o math20.bmi
3 clang++ -std=c++20 -DUSE_STD_FORMAT @util20.modmap -c util.cppm -o util.o -fmodule-output=util20.bmi
4 clang++ -std=c++23 -DUSE_SIMD @math23.modmap -c math.cppm -o math.o -fmodule-output=math23.bmi
5 clang++ -std=c++20 @main20.modmap -c main.cpp -o main.o
6 clang++ math.o util.o main.o -o prog
```

Unsolved Problem in Computer Science #3

```
clang-scan-deps -format=p1689 -std=c++23 -DMATH_EXPORT -c math.cppm -o math.ddi.json
```

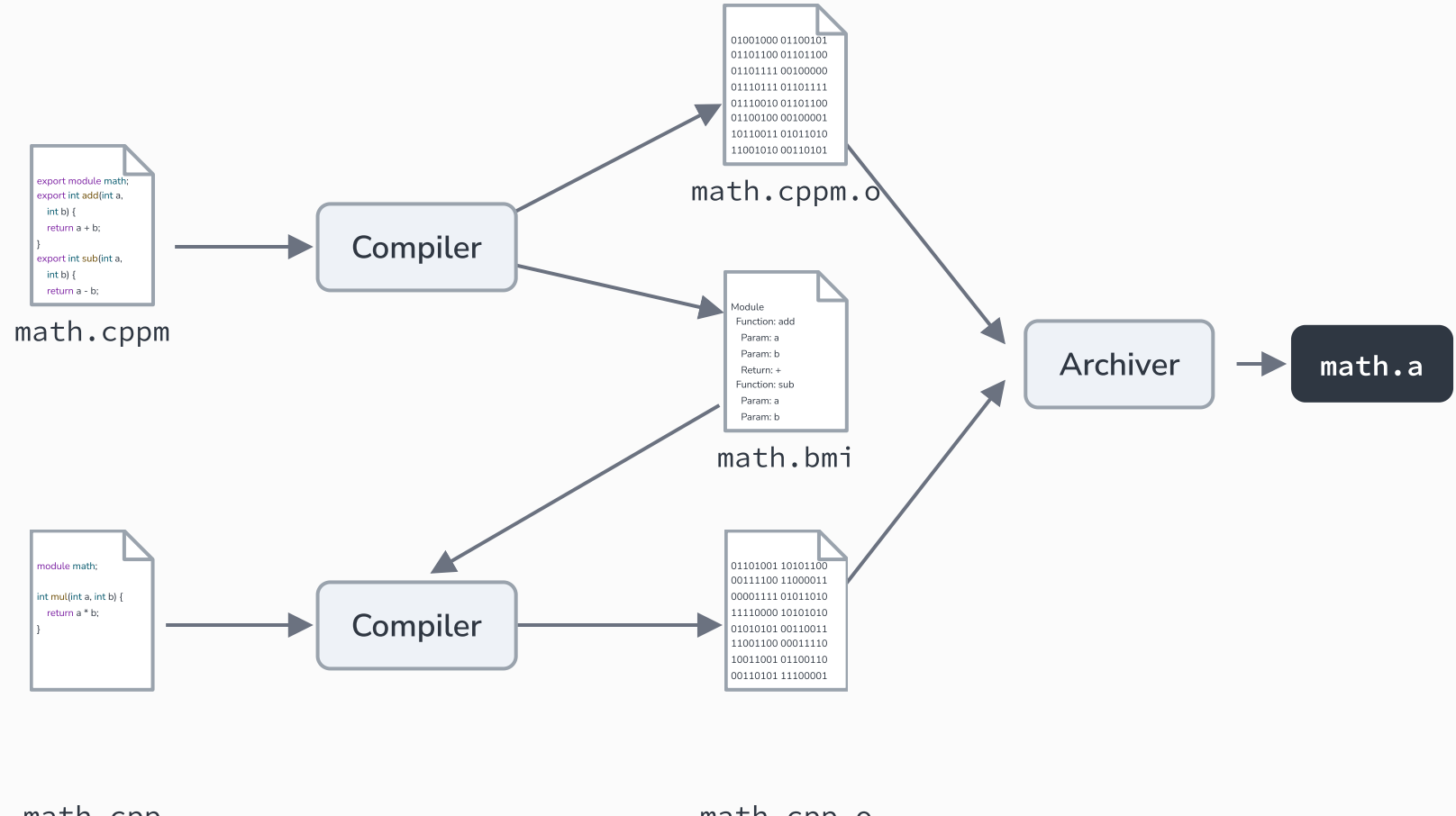
```
clang++ -std=c++20 -DMATH_EXPORT --precompile @math20.modmap math.cppm -o math20.bmi
```

```
clang++ -std=c++23 -DMATH_EXPORT @math23.modmap -c math.cppm -o math.o -fmodule-output=math23.bmi
```

```
#ifdef MATH_EXPORT
#define MATH_API __declspec( dllexport )
#else
#define MATH_API __declspec( dllimport )
#endif

MATH_API int add(int a, int b) {
    return a + b;
}
```

Module Libraries



Module Manifest

```
{  
  "version": 1,  
  "revision": 1,  
  "modules": [{  
    "logical-name": "math",  
    "source-path": "math.cppm",  
    "local-arguments": {  
      "definitions": [{  
        "name": "USE_SIMD"  
      }]  
    }  
  }]  
}
```

CPS Manifest

```
{  
  "name": "math",  
  "cps_version": "0.15.0",  
  "cps_path": "@prefix@/lib/cps/math",  
  "components": {  
    "math": {  
      "type": "archive",  
      "location": "@prefix@/lib/math.a",  
      "cpp_module_metadata": "@prefix@/lib/cps/math.modules.json"  
    }  
  }  
}
```

<https://cps-org.github.io/cps/index.html>

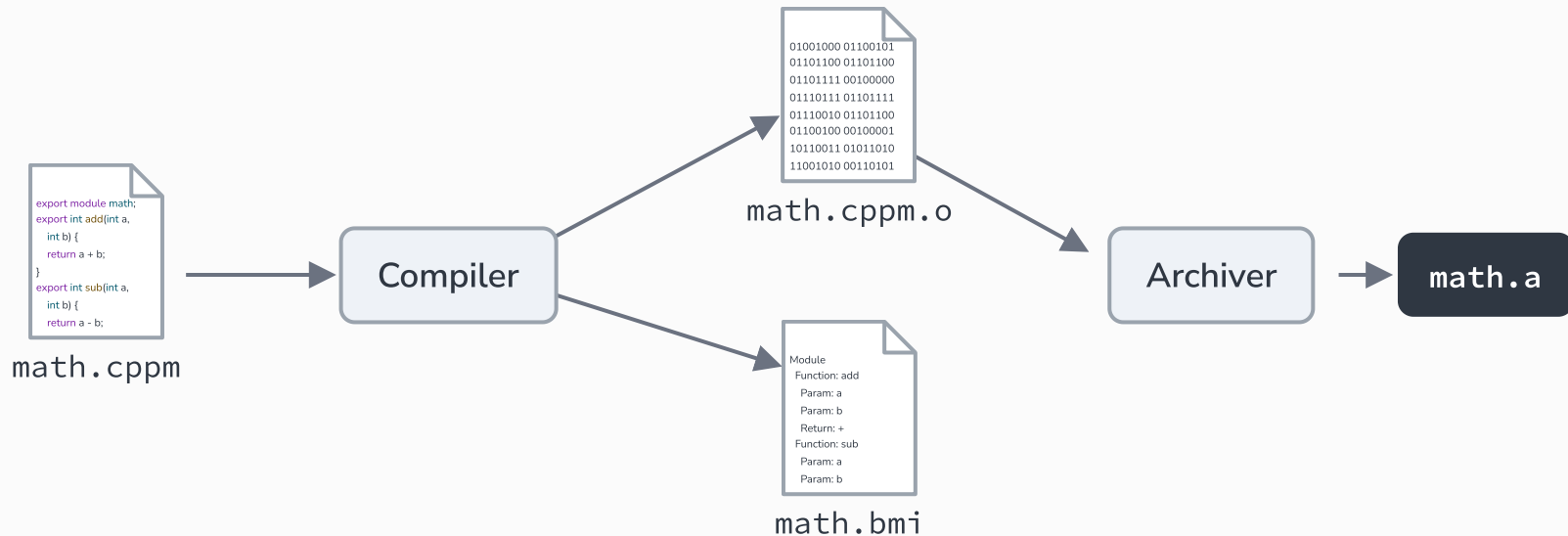
What about the stdlib?

```
{  
  "version": 1,  
  "revision": 0,  
  "library": "microsoft/STL",  
  "module-sources": [  
    "std.ixx",  
    "std.compat.ixx"  
  ]  
}
```

Bonus Slides

Oddities, Gotchas, and Footguns

Interface Only Libraries



Interface Only Libraries

```
export module math;  
  
export template <typename T>  
T add(T a, T b) {  
    return a + b;  
}  
  
export template <typename T>  
T sub(T a, T b) {  
    return a - b;  
}
```

```
$ nm math.o  
000000000000000000 T _ZGIW4math  
    initializer for module math
```

Interface Only Libraries

```
export module math;  
  
export template <typename T>  
T add(T a, T b) {  
    return a + b;  
}  
  
int cppcon = 26;
```

```
import math;  
  
int main() {  
    return add(1, 2);  
}
```

```
$ clang++ -std=c++20 --precompile math.cppm -o math.bmi  
$ clang++ -std=c++20 -fmodule-file=math=math.bmi main.cpp -o prog
```

```
/usr/bin/ld: undefined reference to `initializer for module math'  
clang++: error: linker command failed with exit code 1
```

Bug 124268 - `import std` initializer symbol is not shipped in `libstdc++`**Status:** RESOLVED FIXED**Alias:** None**Product:** gcc**Component:** libstdc++ ([show other bugs](#))**Version:** 15.2.1**Importance:** P3 normal**Target Milestone:** 16.0**Assignee:** Jonathan Wakely**URL:****Keywords:****Depends on:** [124565](#) [124566](#) ~~[124554](#)~~**Blocks:** [c++-modules](#)Show dependency [tree](#)
/ [graph](#)**Reported:** 2026-02-26 19:48 UTC by Ben Boeckel**Modified:** 2026-09-02 16:04 UTC ([History](#))**CC List:** 5 users ([show](#))**See Also:** <https://gitlab.kitware.com/cmake/cmake/-/issues/27626>
https://bugs.freebsd.org/bugzilla/show_bug.cgi?id=277958**Host:****Target:** *-*-darwin**Build:****Known to work:****Known to fail:****Last reconfirmed:** 2026-02-27 00:00:00

Partition Implementation Units

math.cppm

```
export module math;
export import :part;
```

math-part.cppm

```
export module math:part;
export int add(int, int);
```

math-part.cpp

```
module math:part;

int add(int a, int b) {
    return a + b;
}
```

main.cpp

```
import math;

int main() {
    return add(1, 2);
}
```

```
$ clang++ -std=c++20 -c math-part.cppm -o math-part.cppm.o
-fmodule-output=math-part.cppm.bmi
$ clang++ -std=c++20 -fmodule-file=math:part=math-part.cppm.bmi -c
math.cppm -o math.cppm.o -fmodule-output=math.cppm.bmi
$ clang++ -std=c++20 -x c++-module -c math-part.cpp -o math-part.cpp.o
-fmodule-output=math-part.cpp.bmi
$ clang++ -std=c++20 -fmodule-file=math=math.cppm.bmi
-fmodule-file=math:part=math-part.cppm.bmi -c main.cpp -o main.o
```

```
$ clang++ math.cppm.o math-part.cppm.o math-part.cpp.o main.o -o prog
/usr/bin/ld: multiple definition of `initializer for module math:part'
clang++: error: linker command failed with exit code 1
```

Partition Implementation Units

math.cppm

```
export module math;

import :detail;

export point origin();
```

math-detail.cpp

```
module math:detail;

struct point { int x, y; };
```

main.cpp

```
import math;

int main() {
    auto p = origin();
    return 0;
}
```

```
$ clang++ -std=c++20 -x c++-module -c math-detail.cpp -o math-detail.cpp.o
-fmodule-output=math-detail.cpp.bmi
$ clang++ -std=c++20 -fmodule-file=math:detail=math-detail.cpp.bmi -c
math.cppm -o math.cppm.o -fmodule-output=math.cppm.bmi
$ clang++ -std=c++20 -fmodule-file=math=math.cppm.bmi
-fmodule-file=math:detail=math-detail.cpp.bmi -c main.cpp -o main.o
```

main.cpp:4:10: **error**: definition of '**point**' must be imported from module 'math' before it is required

```
4 |     auto p = origin();
  |             ^
```

math-detail.cpp:3:8: note: definition here is **not reachable**

```
3 | struct point { int x, y; };
  |             ^
```

[C++20] [Modules] Clang does not implement strong ownership model when targeting MSVC ABI #89781 ⁴⁶

 Open



huangqinjin opened on Apr 23, 2024

Last edited by VoltrexKeyva ▾

Member



hello.cppm

```
export module hello;
export void greeting() {}
```



MSVC

```
> cl /std:c++20 /TP /interface /c hello.cppm
> dumpbin /symbols hello.obj | findstr greeting
009 00000000 SECT3 notype () External | ?greeting@@YAXXZ::<!hello> (void __cdecl greeting(void)
```





MSVC omits COMDAT symbols for imported inline variables, causing LNK2001

Under Consideration

0



2



9



Votes



Vito Gamberini - Reported Jul 30, 2026 12:20 PM

[severity:I'm unable to use this version]

MSVC does not emit COMDAT symbols for inline variables imported for module interface units. This makes usage of modules containing mutable inline variables via a DLL impossible without rebuilding the interface units locally, which risks exposing ABI information to the consumer.

For example:

```
// provider.cppm
export module provider;
export inline int value = 5;

// Shared libraries must export something
export __declspec(dllexport) void dummy() {}
```

Thank You

Questions?